

V2 Symbol Retriever Handbook

Agentic QA Testing System

Purpose

This handbook explains the design of **V2 Symbol Retriever** for the Agentic QA Testing System.

The goal of V2 is to improve repository retrieval for automated test generation by moving from **file-level keyword retrieval** to **structure-aware symbol retrieval**.

Current workflow context:

```
Requirement
  ↓
Analyze Requirement
  ↓
Retrieve Repository Context
  ↓
Inspect Symbols
  ↓
Explore UI
  ↓
Design Test
  ↓
Generate Test
  ↓
Execute
  ↓
Analyze Result
```

V2 focuses on the **Retrieve Repository Context / Inspect Symbols** part.

1. Why V2 Exists

V1.1: substring keyword retrieval

V1.1 searched repository files with substring matching.

Example:

```
query term: valid
repository text: invalid_password

"valid" in "invalid_password" → True
```

That was wrong.

Another example:

```
query term: register
repository text: registered_user
```

```
"register" in "registered_user" → True
```

This created false scoring evidence.

V1.2: token-aware file retrieval

V1.2 fixed lexical matching.

Instead of:

```
"valid" in "invalid"
```

we tokenize:

```
invalid_password
→ invalid
→ password
```

Now:

```
valid != invalid
register != registered
```

Current 7-case V1.2 RealWorld benchmark:

```
Precision@3 = 0.57
Recall@3    = 0.43
Recall@5    = 0.57
Top1        = 0.86
```

However, V1.2 still retrieves at the **file level**.

For example:

```
pages/editor_page.py
tests/articles/test_edit_article.py
fixtures/articles.py
```

It does not understand individual classes, methods, tests, or fixtures inside those files.

2. Main Limitation of File-Level Retrieval

Consider this requirement:

An authenticated user can edit an existing article.

V1.2 may retrieve:

1. test_edit_article.py
2. editor_page.py
3. test_delete_article.py
4. test_create_article.py
5. article_builder.py
6. article_page.py
7. article_api_setup_smoke.py
8. fixtures/articles.py

The correct test is ranked first.

That looks good.

But code generation needs more than the first file. It needs things such as:

```
test_user_can_edit_article
existing_article fixture
article_update fixture
ArticlePage.edit_article
EditorPage.fill_article_form
EditorPage.publish
```

V1.2 cannot directly retrieve these objects.

It only knows:

```
file + tokens + score
```

V2 should understand:

```
symbol + structure + relationships
```

3. What Is a Symbol?

A **symbol** is a meaningful named code entity.

Examples:

CLASS

```
LoginPage
EditorPage
ArticlePage
```

METHOD

```
LoginPage.sign_in
EditorPage.publish
```

```

        ArticlePage.delete_article

TEST
    test_user_can_edit_article
    test_user_can_delete_article

FIXTURE
    existing_article
    article_update
    registered_user_token

FUNCTION
    build_unique_article

```

Instead of treating this file:

`pages/editor_page.py`

as one retrieval object, V2 can treat it as several:

```

EditorPage
EditorPage.__init__
EditorPage.open_new
EditorPage.open_for_edit
EditorPage.fill_article_form
EditorPage.publish
EditorPage.create_article

```

This gives the retriever much finer granularity.

4. What Is an AST?

AST means **Abstract Syntax Tree**.

Python can parse source code without executing it:

```
import ast
```

```
tree = ast.parse(source_code)
```

Given:

```

class EditorPage(BasePage):
    def publish(self) -> None:
        self.publish_button.click()

```

Python sees something conceptually like:

```
ClassDef
  \-- name = EditorPage
    \-- FunctionDef
      \-- name = publish
```

The AST gives us exact programming-language structure.

It knows:

```
EditorPage is a class
publish is a method
publish belongs to EditorPage
```

This is much more reliable than guessing from plain text.

5. AST vs Symbol Index

These are different concepts.

AST

Temporary parsed representation of Python source.

```
Python source
  ↓
ast.parse()
  ↓
AST
```

Symbol Index

Our own repository representation built from the AST.

```
AST
  ↓
extract meaningful entities
  ↓
RepositorySymbol[]
  ↓
RepositorySymbolIndex
```

We do not retrieve directly from raw AST nodes.

We convert them into our stable domain contracts.

6. Current V2 Contracts

Our V2 models are conceptually:

```
class SymbolType(StrEnum):
    CLASS = "class"
    METHOD = "method"
    FUNCTION = "function"
    TEST = "test"
    FIXTURE = "fixture"
```

A repository symbol:

```
class RepositorySymbol(BaseModel):
    name: str
    qualified_name: str
    symbol_type: SymbolType
    path: str
    parent: str | None = None
    start_line: int
    end_line: int
    source: str
    parameters: list[str]
    decorators: list[str]
    bases: list[str]
```

Example:

```
RepositorySymbol(
    name="publish",
    qualified_name="EditorPage.publish",
    symbol_type=SymbolType.METHOD,
    path="pages/editor_page.py",
    parent="EditorPage",
    start_line=50,
    end_line=52,
    source="def publish(...): ...",
    parameters=["self"],
)
```

7. Important V2 Terms

Symbol — named code entity.

Symbol Type — classification such as CLASS, METHOD, TEST, FIXTURE.

Qualified Name — complete name such as EditorPage.publish.

Parent — containing symbol, e.g. `EditorPage`.

Source Span — exact source lines occupied by the symbol.

Chunk — source text associated with one symbol.

Symbol Index — collection of extracted repository symbols.

Structural Retrieval — retrieval using programming-language structure rather than raw file text.

Dependency Edge — relationship between symbols, such as `TEST → FIXTURE` or `FIXTURE → PAGE OBJECT`.

8. Current AST Indexer

We already implemented:

`PythonAstSymbolIndexer`

Current flow:

`sample_automation/`

↓

`scan *.py files`

↓

`ast.parse()`

↓

`extract:`

`CLASS`
`METHOD`
`FUNCTION`
`TEST`
`FIXTURE`

↓

`RepositorySymbolIndex`

The indexer extracts:

`name`
`qualified_name`
`symbol_type`
`path`
`parent`
`parameters`
`decorators`
`bases`
`start_line`

```
end_line
source
```

9. Why Parameters Matter

Consider:

```
def test_user_can_edit_article(
    api_authenticated_user: UserData,
    existing_article: CreatedArticle,
    article_update: ArticleData,
    article_page: ArticlePage,
    editor_page: EditorPage,
) -> None:
```

The AST index stores:

```
api_authenticated_user
existing_article
article_update
article_page
editor_page
```

This is valuable structural information.

Later we can resolve those parameter names into real fixture or object relationships.

10. Why Decorators Matter

Example:

```
@pytest.fixture
def existing_article(...):
```

Without the decorator this looks like a regular function.

With:

```
decorator = pytest.fixture
```

we classify it as:

```
FIXTURE
```

11. Why Class Bases Matter

Example:

```
class EditorPage(BasePage):
```

AST gives:

```
bases = ["BasePage"]
```

Later this can help identify Page Object structure:

```
BasePage
  ↑
  +-- LoginPage
  +-- RegisterPage
  +-- EditorPage
  \-- ArticlePage
```

12. V2 Symbol Retriever: Core Architecture

V2 should separate two responsibilities.

Indexing

Question:

What exists in the repository?

Component:

PythonAstSymbolIndexer

Output:

RepositorySymbolIndex

Retrieval

Question:

Which symbols are relevant to this requirement?

Component:

SymbolRepositoryContextProvider

Output:

SymbolRetrievalContext

Architecture:

```
Repository
  ↓
PythonAstSymbolIndexer
  ↓
RepositorySymbolIndex
  ↓
Symbol Retriever
  ↓
RetrievedSymbol[]
```

Do not combine indexing and retrieval into one giant class.

13. First V2 Retrieval Algorithm

The first V2 version should remain deliberately simple.

We want a clean experiment:

V1.2
file-level + token matching

vs

V2.0
symbol-level + token matching

Do not add embeddings, LLM reranking, semantic similarity, or graph traversal yet.

14. Step 1: Build Query Terms

We continue to use RequirementAnalysis.

Example:

```
feature: Article management
operation: Edit article
conditions:
- user is authenticated
- existing article is available
expected outcomes:
- editor loads existing values
- article can be updated
```

This becomes concepts such as:

article
edit
authenticated
existing
editor
loads
updated
values

For the first V2 experiment, query construction should stay close to V1.2.

15. Step 2: Score Symbols Instead of Files

Example symbol:

qualified_name:
test_user_can_edit_article

type:
TEST

parameters:
api_authenticated_user
existing_article
article_update
article_page
editor_page

Tokenized representation contains concepts like:

test
user
can
edit
article
api
authenticated
existing
article
update
page
editor

A delete test has:

test

```
user
can
delete
article
```

For an edit query, the edit test should naturally score higher.

16. What Text Should Be Scored?

Useful fields include:

```
name
qualified_name
parameters
parent
source
path
decorators
bases
```

A symbol such as:

```
EditorPage.publish
```

can be represented using:

```
qualified_name: EditorPage.publish
parent: EditorPage
source: def publish(...): ...
path: pages/editor_page.py
```

17. Weighted Structural Evidence

A future scoring formula might use:

symbol-name match	high weight
qualified-name match	high weight
parameter match	medium weight
parent match	medium weight
path match	medium weight
source match	low weight

Example conceptual weights:

symbol name	+8
qualified name	+8

```
parameters      +4
path             +3
source tokens    +1..3
```

For the first implementation, keep scoring simple enough to understand.

18. Example: Edit Article

Requirement:

Authenticated article author edits an existing article.

Potential V2 candidates:

TEST

test_user_can_edit_article

METHOD

ArticlePage.edit_article

CLASS

ArticlePage

CLASS

EditorPage

METHOD

EditorPage.fill_article_form

METHOD

EditorPage.publish

FIXTURE

existing_article

FIXTURE

article_update

This is much more useful than only retrieving files.

19. Example: Delete Article

Requirement:

Authenticated author deletes an existing article.

Potential symbols:

TEST

test_user_can_delete_article

METHOD

ArticlePage.delete_article

FIXTURE

existing_article

FIXTURE

registered_user_token

FIXTURE

realworld_api_client

METHOD

RealWorldApiClient.article_exists

This closely matches the implementation structure.

20. Example: Login

For login, V2 can distinguish:

TEST

test_registered_user_can_sign_in

METHOD

LoginPage.sign_in

FIXTURE

registered_user

METHOD

Navigation.user_profile_link

from registration-related symbols such as:

RegisterPage.register

test_user_can_register

because symbol identity is more specific than file vocabulary.

21. RetrievedSymbol Contract

A repository symbol is a fact.

A retrieved symbol is a query-specific result.

```
class RetrievedSymbol(BaseModel):
    symbol: RepositorySymbol
    score: float
    matched_terms: list[str]
```

So:

```
RepositorySymbol
    = repository truth
```

```
RetrievedSymbol
    = relevance for one requirement
```

22. SymbolRetrievalContext

Example:

```
class SymbolRetrievalContext(BaseModel):
    query_terms: list[str]
    items: list[RetrievedSymbol]
```

Possible result:

1. test_user_can_edit_article
 2. ArticlePage.edit_article
 3. EditorPage.fill_article_form
 4. existing_article
 5. EditorPage.publish
-

23. Evaluation Compatibility

Our current LangSmith ground truth is file-based:

```
tests/articles/test_edit_article.py
pages/article_page.py
pages/editor_page.py
```

```
fixtures/articles.py
data/article_builder.py
```

But V2 returns symbols.

For the first comparison, collapse ranked symbols into unique file paths.

Example:

```
1 test_user_can_edit_article
  → tests/articles/test_edit_article.py

2 ArticlePage.edit_article
  → pages/article_page.py

3 ArticlePage
  → pages/article_page.py

4 existing_article
  → fixtures/articles.py

5 EditorPage.fill_article_form
  → pages/editor_page.py
```

Collapse duplicates:

```
1 tests/articles/test_edit_article.py
2 pages/article_page.py
3 fixtures/articles.py
4 pages/editor_page.py
```

Then the same evaluator can calculate:

```
Precision@3
Recall@3
Recall@5
Top1
```

24. Why Collapse Duplicate Paths?

Without collapsing:

```
ArticlePage
ArticlePage.edit_article
ArticlePage.delete_article
```

could occupy several top positions even though they all come from one file.

That would make comparison with file-level V1.2 unfair.

So:

```
symbol ranking
  ↓
unique-path projection
  ↓
existing evaluator
```

25. What Success Looks Like

Current V1.2 enlarged benchmark:

```
P@3   = 0.57
R@3   = 0.43
R@5   = 0.57
Top1  = 0.86
```

The goal is not only to improve Top1.

For Agentic QA, context completeness is especially important:

```
Recall@3 ↑
Recall@5 ↑
```

because generation needs the surrounding test, Page Objects, fixtures, builders, and helpers.

26. V2.0 Scope

Initial V2 should include:

```
CLASS
METHOD
FUNCTION
TEST
FIXTURE
```

with:

```
name
qualified_name
path
parent
source
start_line
end_line
```

parameters
decorators
bases

Retrieval:

token-aware symbol ranking

Evaluation:

symbol results
→ unique file paths
→ existing metrics

27. What V2.0 Should NOT Include

Do not add yet:

embeddings
vector database
semantic RAG
LLM reranking
full call graph
complex dependency graph
locator indexing
Playwright MCP
automatic code generation

This keeps the first experiment interpretable.

28. Future Enhancement: Dependency Resolution

Current AST stores:

TEST test_user_can_edit_article

parameters:
 api_authenticated_user
 existing_article
 article_update
 article_page
 editor_page

Future dependency resolution can connect those names to real indexed symbols:

```

test_user_can_edit_article
    +-- USES_FIXTURE → api_authenticated_user
    +-- USES_FIXTURE → existing_article
    +-- USES_FIXTURE → article_update
    +-- USES_FIXTURE → article_page
    \-- USES_FIXTURE → editor_page

```

29. Future Enhancement: Fixture Return Types

Example:

```

@pytest.fixture
def article_page(...) -> ArticlePage:

```

Future metadata:

```

return_type = ArticlePage

```

Relationship:

```

article_page fixture
    ↓ RETURNS
ArticlePage

```

30. Future Enhancement: Call Relationships

Inside the edit test:

```

article_page.edit_article()
editor_page.fill_article_form(article_update)
editor_page.publish()

```

Future AST analysis can build:

```

test_user_can_edit_article
    +-- CALLS → ArticlePage.edit_article
    +-- CALLS → EditorPage.fill_article_form
    \-- CALLS → EditorPage.publish

```

31. Future Enhancement: Import Relationships

Example:

```

from sample_automation.pages.article_page import ArticlePage

```

Potential relationship:

```
test_edit_article.py
  ↓ IMPORTS
ArticlePage
```

32. Future Enhancement: Locator Symbols

Later we may index assignments such as:

```
self.publish_button = page.get_by_role(
    "button",
    name="Publish Article",
)
```

Potential symbol:

```
LOCATOR
EditorPage.publish_button
```

Possible metadata:

```
strategy = get_by_role
role = button
name = Publish Article
```

Useful later for code generation and MCP verification.

33. Future Enhancement: Component Relationships

Example:

```
self.navigation = Navigation(page)
```

Potential relationship:

```
ArticlePage
  ↓ COMPOSES
Navigation
```

This helps the retriever understand reusable UI composition.

34. Future Enhancement: Symbol-Level Evaluation

Current ground truth is file based:

`pages/editor_page.py`

Future ground truth can be symbol based:

`EditorPage.fill_article_form`

`EditorPage.publish`

Possible metrics:

Symbol Precision@K

Symbol Recall@K

Symbol Top1

Required-symbol coverage

35. Future Enhancement: Semantic RAG

AST is exact but not semantic.

It does not automatically understand:

`remove ~ delete`

`post ~ article`

`sign in ~ authenticate`

Semantic RAG can handle those similarities.

Future architecture:

Requirement

↓

semantic retrieval

+

symbol retrieval

↓

hybrid candidate set

Responsibilities:

RAG

→ semantic knowledge

AST

→ exact code structure

LLM
→ reasoning

Playwright MCP
→ live application state

36. Future Enhancement: Hybrid Retrieval

A mature retriever may combine:

```
token score
+
symbol-name score
+
structural relationship score
+
semantic embedding score
+
artifact-type signal
```

But each additional signal should be justified by evaluation.

37. Future Enhancement: Incremental Indexing

Current indexing can scan the repository each time.

For larger repositories:

```
Git change
↓
changed files only
↓
reparse changed symbols
↓
update index
```

Useful metadata:

```
path
file hash
git revision
modified time
```

38. Future Enhancement: Persistent Index

Current V2 can keep symbols in memory.

Later storage options may include:

SQLite
PostgreSQL
document store
vector database
hybrid index

Keep contracts independent of storage technology.

39. Future Enhancement: Repository Graph

Eventually the repository can be represented as a graph:

TEST
↓ uses
FIXTURE
↓ returns
PAGE OBJECT
↓ contains
METHOD
↓ uses
COMPONENT

Then retrieval can become:

find strong seed symbols
↓
expand related neighbors
↓
rank combined context

40. Future Enhancement: Retrieval Diversity

A symbol retriever may return many symbols from one file:

ArticlePage
ArticlePage.__init__
ArticlePage.title
ArticlePage.body

```
ArticlePage.edit_article
ArticlePage.delete_article
```

Later we may introduce diversity rules such as:

```
maximum N symbols per file
```

or coverage across several artifact roles.

Only add this if evaluation shows a need.

41. Future Enhancement: Artifact-Type Awareness

Future ranking may favor different symbol types depending on context.

Example:

```
edit article
```

could prefer:

```
TEST
METHOD
FIXTURE
```

over unrelated helper symbols.

Avoid hardcoding too many assumptions without measurement.

42. Relationship to LangGraph

V2 fits the orchestration cleanly:

```
analyze_requirement
  ↓
retrieve_context
  ↓
inspect_symbols
  ↓
explore_ui
  ↓
design_test
```

Possible future separation:

```
retrieve_context
  → semantic/file retrieval
```

```
inspect_symbols
  → exact AST lookup
```

Keep node responsibilities explicit.

43. Relationship to Test Generation

The generator should eventually receive structured context such as:

```
Relevant existing test:
  test_user_can_edit_article
```

```
Required Page Objects:
  ArticlePage
  EditorPage
```

```
Relevant methods:
  ArticlePage.edit_article
  EditorPage.fill_article_form
  EditorPage.publish
```

```
Fixtures:
  existing_article
  article_update
  api_authenticated_user
```

This is much safer than giving the LLM whole files and asking it to guess what to reuse.

44. Relationship to Hallucination Prevention

Bad generated code:

```
article_page.update_article(...)
```

when no such method exists.

V2 can explicitly expose:

```
ArticlePage.edit_article
EditorPage.fill_article_form
EditorPage.publish
```

Then generation can be constrained to existing symbols unless a new abstraction is explicitly proposed.

45. Proposed V2 Implementation Order

Phase 1 — complete:

contracts
AST indexer
unit tests
manual inspection

Phase 2 — next:

SymbolRepositoryContextProvider

Responsibilities:

accept RequirementAnalysis
build query terms
score RepositorySymbol objects
return SymbolRetrievalContext

Phase 3:

project symbols → unique paths

for compatibility with the current evaluator.

Phase 4:

Run the exact same 7 RealWorld cases.

Compare against:

V1.2 baseline

P@3 0.57

R@3 0.43

R@5 0.57

Top1 0.86

Phase 5:

Diagnose per-case behavior before tuning.

Phase 6:

Only if justified:

dependency resolution
symbol-level evaluation

46. First V2 Experiment

Keep:

```
same RequirementAnalysis
same 7 cases
same repository revision
same query concepts
same tokenizer
same evaluation metrics
same top-k path comparison
```

Change only:

V1.2:

```
file is retrieval unit
```

V2:

```
symbol is retrieval unit
```

This is the main experimental variable.

47. What We Hope to Learn

The research question is:

Does symbol-level structural granularity improve repository context
retrieval for automated test generation?

If V2 improves Recall@3 / Recall@5 while maintaining strong Top1, symbol
retrieval is adding measurable value.

If it does not, diagnose why before adding complexity.

48. Architectural Principle

The most important principle is:

Do not ask the LLM to rediscover repository structure
that Python can determine exactly.

AST can determine:

class
method
fixture
test
parameter
decorator
source location
inheritance

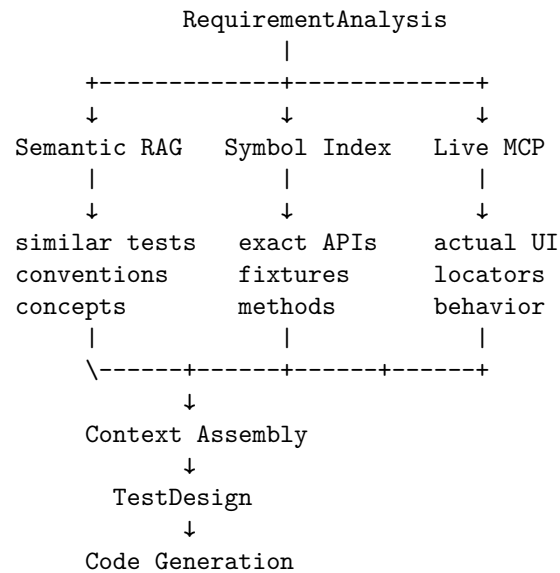
deterministically.

Use the LLM for:

reasoning
semantic interpretation
test design
generation
failure analysis

Use AST for exact code facts.

49. Long-Term Architecture



Each subsystem has a distinct responsibility.

50. Summary

V1.2 retrieves files:

```
pages/editor_page.py
fixtures/articles.py
tests/articles/test_edit_article.py
```

V2 retrieves symbols:

```
EditorPage.fill_article_form
EditorPage.publish
existing_article
article_update
test_user_can_edit_article
```

Main benefits:

```
finer retrieval granularity
exact code structure
better reuse discovery
less irrelevant context
better fixture/Page Object discovery
lower risk of invented APIs
foundation for dependency graph retrieval
```

The first V2 implementation should remain intentionally simple:

```
AST symbol index
+
token-aware symbol ranking
+
existing path-level evaluation
```

Then evaluate.

Only after we have evidence should we add:

```
dependency edges
return types
method calls
imports
locator indexing
symbol-level metrics
semantic RAG
hybrid ranking
persistent indexing
```

That keeps the Agentic QA architecture measurable, understandable, and maintainable.